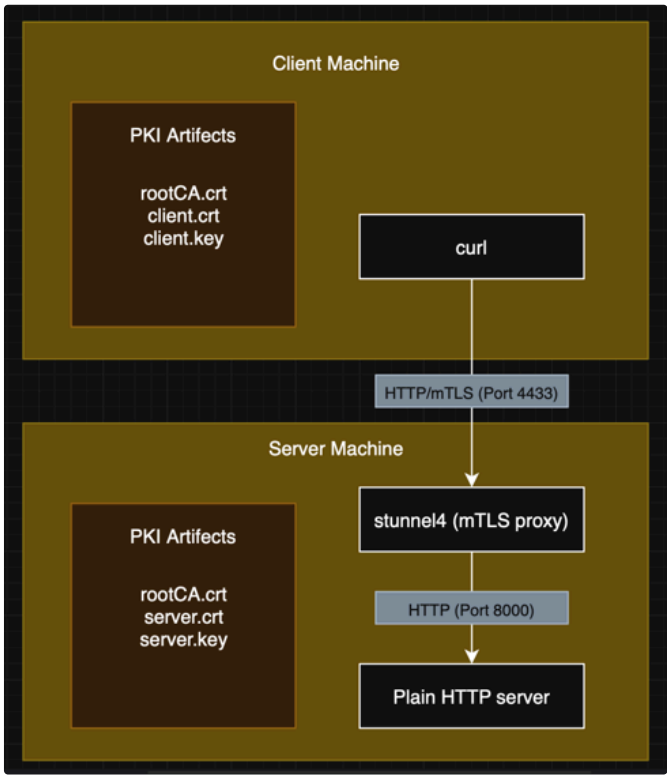
[2026 NS] TLS Connection with Bidirectional Certificates

Introduction

In this assignment, you will gain hands-on experience establishing an mTLS connection for a plaintext service using `stunnel4`. `stunnel4` is used to wrap a service lacking native encryption support in a secure mTLS layer. Acting as an mTLS proxy, `stunnel4` enforces strict client certificate verification against the Root CA and forwards the connection to the actual service only after successful authentication.



Objectives

- Experience the Public Key Infrastructure (PKI) lifecycle by generating a custom Root CA, server certificate, and client certificate.
- Secure a plaintext backend service by configuring `stunnel4` as an mTLS proxy.
- Analyze and verify the mTLS handshake process at both the application layer (`curl`) and the network layer (packet capture).

Turn In

- Experiment report (`yourStudentID_report.pdf`)

Requirements

- **Environment:** You must use two separate virtual machines (VMs) or containers to simulate the Client and Server.
- **PKI Setup:** Use the `openssl` command-line tool to create a self-signed Root CA. Use this CA to sign two separate certificates: one for the Server and one for the Client.
- **Server Setup:**
  - Start a plaintext backend service using `python3 -m http.server 8000`.
  - Configure `stunnel4` to listen on port 4433, forward traffic to localhost:8000, and strictly require client certificate verification.
- **Verification:** Use `curl` on the Client machine to test the connection. Capture the network traffic using Wireshark or `tcpdump` during a successful connection.

Report Specification & Grading

Format: Submit a single PDF file containing your configurations, command logs, and annotated screenshots.

Part 1: PKI Construction (30 pts)

List all `openssl` commands used to generate the CA, keys, CSRs (Certificate Signing Requests), and Certificates.

Part 2: Service Configuration (20 pts)

Provide the exact content of your `stunnel.conf` file.

Part 3: Connection Testing (30 pts)

Provide screenshots of the `curl -v` (verbose) output for the following three scenarios:

1. **Success:** Connection using the valid client certificate and CA (`curl -v --cacert ... --cert ... --key ...`). Show the HTTP 200 OK response.
2. **Failure 1:** Connection without providing a client certificate. Show the TLS alert/handshake failure.
3. **Failure 2:** Connection using an unauthorized client certificate (generate a fake cert signed by a different, untrusted CA). Show the server rejecting the connection.

Part 4: Packet Analysis (20 pts)

Provide a Wireshark or `tcpdump` screenshot of the network traffic during the successful mTLS handshake (Scenario 1).

For questions, please contact TA Mr. Chen <shflte1314527@gmail.com>